

From Karaka Roles to Executable Isolation Semantics

A Reproducible RV32 Case Study of a Paninian-Inspired Software Architecture

Prepared for: Praveen Kumar **Project:** Pāṇinian Systems Language (PSL) **Study date:** 23 May 2026 **Status:** Technical case study based on reproducible prototype evidence

Abstract

This case study investigates a narrow but consequential question: can concepts inspired by Paninian grammatical analysis serve as operational programming semantics rather than merely as linguistic annotation or natural-language processing metadata? The PSL prototype maps selected Karaka-inspired roles to device targets, selected action terms to opcodes, and a Siddha-Asiddha-inspired visibility distinction to visible versus shadow state. A formal compiler frontend emits a fixed-width 32-bit binary instruction image. A hosted C reference model first verified decoder behavior. The study then advanced the same compiled image into freestanding RV32 firmware, linked at 0x80000000 and executed under QEMU full-system RISC-V emulation on the virt machine. The firmware emitted trace evidence through the emulated UART MMIO device at 0x10000000.

The result is affirmative at the prototype level: a Paninian-inspired semantic vocabulary was used to write a source program whose compiled execution deterministically committed visible state, isolated shadow state, rejected an unauthorized protected target, and admitted an authorized protected transaction. This is evidence of software utility for a defined formal subset. It is not yet evidence that the full Ashtadhyayi is a general-purpose programming language, that PSL outperforms established architectures, or that the design has been realized in physical silicon or HDL.

1. Research Problem

Pāṇinian scholarship has been repeatedly examined for computational applications, especially formal analysis, parsing, morphological generation, and knowledge representation. Briggs (1985) argued that Sanskrit grammatical analysis bears a close formal relationship to knowledge-representation schemata in artificial intelligence. Later computational-linguistics work investigated how aspects of Pāṇinian grammar may be modelled computationally.

PSL asks a different engineering question:

Can a carefully bounded, Paninian-inspired semantic system act as the source language for executable state and access-control behavior in a computer runtime?

The word “inspired” is essential. This prototype does not claim to implement the entire grammatical tradition or establish an equivalence between Sanskrit and machine code. It tests whether selected conceptual distinctions can be made operational and reproducible.

2. Contribution and Claim Boundary

The implemented contribution consists of four parts:

Contribution	Implemented Result	Claim Boundary
Source vocabulary	Selected Devanagari program terms map to targets, actions, and privilege metadata.	A defined DSL subset, not natural-language understanding.
Binary contract	Compiler emits packed, big-endian 32-bit instruction words.	A stable prototype ABI, not a standardized ISA.
Visibility model	Siddha targets commit to visible state; Asiddha targets write to shadow state; Yantra requires Ring 0.	A reference access-control model, not MMU replacement.
RV32 proof	Freestanding firmware executes the compiled image under QEMU virt and reports by UART MMIO.	System-emulated hardware behavior, not physical silicon.

Accordingly, the defensible claim is:

PSL demonstrates that a bounded Pāṇinian-inspired semantic mapping can be used to author, compile, and execute a software program with deterministic visibility and privilege behavior on a freestanding RV32 target under full-system emulation.

3. Paninian-Inspired Semantic Mapping

PSL borrows a small number of organizing ideas and gives them precise software meanings. These are design mappings, not assertions that the traditional grammatical terms originally denoted computer hardware.

PSL Concept	Software Meaning in This Prototype	Example
Karman-inspired target	Destination device coordinate	वाचम् maps to 0x30
Kriya-inspired action	Mutating operation code	लिखति maps to write opcode 0x05
Vrddhi-inspired authorization	Kernel-clearance marker in ABI	यन्त्रै selects Ring 0x0

	ring bits	
Siddha	Immediately visible device-state region	Targets below 0x50
Asiddha	Shadow-state region not immediately visible globally	Targets at or above 0x50

The proof program is intentionally small:

```
तन्त्रशास्त्रम् {
  वाचम् लिखति ।
  श्रोत्रम् लिखति ।
  यन्त्रम् लिखति ।
  यन्त्रे स्थापयति ।
}
```

It is designed to exercise four distinct outcomes: visible write, isolated write, rejected protected write, and accepted privileged write.

4. Executable Architecture

4.1 Fixed-Width ABI

Every executable statement produces one 32-bit instruction:

```
[RING:4][COMPRESSION:4][OPCODE:8][TARGET:8][FLAGS:8]
```

For the experimental program, compilation produced:

Source Intent	ABI Word	Expected Runtime Behavior
Visible Vāk write	0x2005300F	global[0x30] = 0xFF
Hidden Śrotra write	0x2005500F	shadow[0x50] = 0xFF; global remains zero
Unprivileged Yantra write	0x2005600F	rejected
Privileged Yantra store	0x00CC600F	shadow[0x60] = 0xFF; global remains zero

4.2 Compiler and Firmware Path

The validated path is:

```
isolation_core.pvm
-> sanskrit_compiler.py
-> pvm_launch.bin (four packed ABI words)
-> embed_binary_image.py
-> pvm_image.h
-> pvm_firmware.c + start.S + linker.ld
```

```
-> pvm_rv32.elf
-> qemu-system-riscv32 -M virt -bios none
```

This path matters because the RV32 firmware does not use hosted file I/O or Linux system calls. The compiler output is included as firmware data and decoded after firmware reset.

5. Experimental Method

5.1 Toolchain

Item	Value
Cross-compiler	riscv64-unknown-elf-gcc (14.2.0+19) 14.2.0
Emulator	QEMU emulator version 10.2.1
Emulated platform	QEMU RISC-V virt board, RV32 system emulation
Boot mode	-bios none with a freestanding ELF
Runtime language	Freestanding C plus RV32 startup assembly

5.2 Hardware Map Verification

QEMU documents virt as a generic RISC-V virtual platform with an NS16550 UART and a SiFive test device. For this experiment, the automatically generated device tree was extracted and inspected before executing firmware. It reported:

Device	Device-Tree Address	Use in Experiment
RAM	0x80000000	Firmware text, read-only image, and mutable PSL state
Serial UART	0x10000000	Observable firmware trace via volatile MMIO write
SiFive test device	0x00100000	Clean test completion signal

5.3 Reproducibility Procedure

The executable test command is:

```
$env:PVM_VM_PASSWORD = "<vm-password>"
python -B run_rv32_system_pipeline.py
```

The runner transfers source artifacts to the Lubuntu VM, rebuilds the packed binary image, embeds it in firmware, compiles the RV32 ELF, extracts the live device tree, runs QEMU system emulation, and asserts the expected summary.

6. Results

6.1 Firmware Placement

The compiled firmware was linked into the QEMU virt RAM range:

Section	Virtual Address	Size
.text	0x80000000	0x446 bytes
.rodata	0x80000448	0x14D bytes
.bss	0x80000598	0x208 bytes

The firmware explicitly clears .bss at reset, then executes the embedded PSL image, ensuring that unchanged visible-state values are testable results rather than assumptions about initialization.

6.2 Runtime Trace

The UART-observed system-emulation output was:

```
[RV32 PVM BOOT] Packed ABI image executing on QEMU virt UART MMIO
[RV32 PVM] word=0x2005300F target=0x30
  SIDDHA global=0xFF
[RV32 PVM] word=0x2005500F target=0x50
  ASIDDHA shadow=0xFF global=0x00
[RV32 PVM] word=0x2005600F target=0x60
  FAULT: YANTRA requires Ring 0
[RV32 PVM] word=0x00CC600F target=0x60
  ASIDDHA shadow=0xFF global=0x00
[RV32 PVM SUMMARY] VAK global=0xFF SROTRA global=0x00 shadow=0xFF YANTRA global=0x00
shadow=0xFF rejected=0x01
```

6.3 Interpretation

The experiment verified all four expected outcomes:

Tested Property	Expected	Observed	Result
Siddha visibility	Vāk committed to global state	global[0x30]=0xFF	Pass
Asiddha isolation	Śrotra shadow updated, global unchanged	shadow=0xFF, global=0x00	Pass
Privilege rejection	Non-kernel Yantra write rejected	rejected=0x01	Pass
Privileged mutation	Ring 0 Yantra store reaches shadow state	shadow[0x60]=0xFF	Pass

The experiment therefore establishes executable usefulness for the selected semantic mapping: the terms used in the PSL source program are not comments or decorative syntax; they determine generated instruction fields and observable firmware behavior.

7. Why This Matters

Previous work cited here demonstrates that Pāṇinian analysis has been valuable in knowledge representation and in computational models of language. PSL explores an adjacent but different direction: using grammatical-role-inspired distinctions to specify operational behavior in a compact program.

The contribution is not that Sanskrit replaces all conventional systems software. The contribution is a reproducible demonstration that:

1. A linguistically motivated role system can provide a structured source notation for stateful computation.
2. The notation can compile into a fixed binary ABI rather than remaining interpretive prose.
3. Visibility and authorization distinctions can be tested as executable properties on a freestanding target.

This creates a credible foundation for further comparative research: source clarity, correctness properties, static validation, code density, formal verification, and hardware-decoder feasibility.

8. Limitations and Research Discipline

The following claims are **not** established by this experiment:

- It does not implement the full Aṣṭādhyāyī or full Sanskrit grammar.
- It does not show that a Paninian-inspired language is superior to C, Rust, capability architectures, or conventional policy-enforcement mechanisms.
- It does not replace an operating-system scheduler or MMU.
- It does not execute on physical RISC-V hardware.
- It does not establish an HDL, FPGA, ASIC, patent, or first-in-field claim.

These boundaries are necessary for credible scholarship. The present result is a reproducible prototype demonstration and a starting point for evaluation.

9. Next Research Milestones

Milestone	Required Evidence	Research Value
Larger DSL coverage	Validated compiler cases for inheritance, elision, precedence, and errors	Shows the approach scales beyond four statements
Formal properties	Tests or model checking for non-interference and privilege	Converts metaphor into

	invariants	enforceable correctness claims
Comparative benchmark	Equivalent C/Rust policy programs and measured binary/runtime properties	Tests whether the approach offers practical benefit
Physical-board port	Named RV32 board, linker/MMIO map, captured UART or GPIO results	Establishes execution beyond system emulation
HDL decoder	Synthesizable RTL plus simulation/formal traces	Tests the hardware-decoder research thesis
Peer review	Public repository, preprint/workshop submission, independent reproduction	Creates external technical validation

10. Conclusion

This case study reports a bounded, reproducible success. PSL has crossed from a conceptual narrative into executable evidence: a Pāṇinian-inspired program is compiled into fixed-width binary instructions and executed by freestanding RV32 firmware under full-system emulation, where its semantics determine visible state, shadow isolation, and privilege rejection.

The result does not prove every larger ambition of the project. It does prove something worthy of serious further work: grammatical concepts can be operationalized as disciplined software semantics and validated at the firmware level. The next phase should focus on formal verification, comparative benchmarks, independent replication, and publication-quality review.

References

- Briggs, Rick. “Knowledge Representation in Sanskrit and Artificial Intelligence.” *AI Magazine* 6, no. 1 (1985): 32. DOI: 10.1609/aimag.v6i1.466.
- Scharf, Peter M. “Modeling Pāṇinian Grammar.” In *Sanskrit Computational Linguistics: Revised Selected and Invited Papers*, Lecture Notes in Artificial Intelligence 5402, Springer, 2009.
- QEMU Project. “‘virt’ Generic Virtual Platform (virt).” QEMU System Emulator Documentation, accessed 23 May 2026.
- QEMU Project. “RISC-V System Emulator.” QEMU System Emulator Documentation, accessed 23 May 2026.

Appendix A: Reproducible Artifact Index

Artifact	Purpose
isolation_core.pvm	Paninian-inspired source proof program

src/utls/sanskrit_compiler.py	Formal subset compiler and 32-bit ABI emitter
src/utls/build_firmware.py	Binary image build entry point
src/utls/embed_binary_image.py	Embeds ABI binary into freestanding firmware
src/rv32/start.S	RV32 reset entry and .bss initialization
src/rv32/linker.ld	Firmware memory layout at QEMU virt RAM base
src/rv32/pvm_firmware.c	MMIO UART firmware decoder and state model
run_rv32_system_pipeline.py	Self-checking remote build and execution runner
evidence/2026-05-23-rv32-qemu-virt-proof.md	Captured experimental evidence record